

GymIO

Documentação Técnica

Instalação · Configuração · Execução do Projeto
Backend · Frontend · Supabase

Versão 1.0 · 2025

1. Introdução

Este documento apresenta os procedimentos necessários para instalação, configuração e execução do sistema **GymIO**, contemplando backend e frontend, bem como a integração com o serviço de banco de dados **Supabase**.

2. Instalação de Dependências

O projeto possui dependências distintas para backend e frontend, sendo necessário o ambiente **Node.js (versão 16 ou superior)** com gerenciador de pacotes npm.

2.1 Backend (Node.js)

A instalação das dependências do backend é realizada por meio dos seguintes comandos no terminal:

```
cd /home/alunos/a2539217/Documentos/GitHub/GymIO/backend  
npm install
```

Bibliotecas instaladas:

- **Express** — framework para APIs REST
- **cors** — controle de acesso
- **dotenv** — gerenciamento de variáveis de ambiente
- **@supabase/supabase-js** — integração com Supabase

Caso o arquivo **package.json** não esteja previamente configurado:

```
npm init -y  
npm install express cors dotenv @supabase/supabase-js
```

2.2 Frontend (React com Vite)

Para instalação das dependências do frontend:

```
cd /home/alunos/a2539217/Documentos/GitHub/GymIO/frontend  
npm install
```

Bibliotecas instaladas:

- React
- React DOM
- React Router DOM
- Lucide React (ícones)
- Vite
- **@vitejs/plugin-react**

2.3 Verificação de Dependências

```
npm list
```

Em caso de inconsistências ou vulnerabilidades:

```
npm audit fix
```

3. Configuração do Banco de Dados (Supabase)

O projeto utiliza o **Supabase** como solução de backend-as-a-service, fornecendo banco de dados PostgreSQL, autenticação e APIs automáticas.

3.1 Pré-requisitos

- Conta ativa no Supabase
- Projeto criado na plataforma
- URL do projeto e chave pública (anon key)

3.2 Configuração da Conexão

No backend, crie um arquivo **supabaseClient.js** com o conteúdo:

```
const { createClient } = require("@supabase/supabase-js");

const supabase = createClient(
  process.env.SUPABASE_URL,
  process.env.SUPABASE_KEY
);

module.exports = supabase;
```

Defina as variáveis no arquivo **.env**:

```
SUPABASE_URL=URL_DO_PROJETO
SUPABASE_KEY=CHAVE_PUBLICA
```

3.3 Script de Criação das Tabelas

Execute o seguinte script no **SQL Editor** do Supabase:

```

CREATE TABLE pessoa (
  id          SERIAL PRIMARY KEY,
  nome        TEXT,
  cpf         TEXT,
  telefone    TEXT,
  email       TEXT,
  endereco    TEXT,
  data_nascimento DATE,
  password    TEXT
);

CREATE TABLE aluno (
  id          SERIAL PRIMARY KEY,
  pessoa_id   INTEGER,
  matricula   TEXT,
  senha       TEXT
);

CREATE TABLE plano (
  id          SERIAL PRIMARY KEY,
  nome        TEXT
);

CREATE TABLE assinatura (
  id          SERIAL PRIMARY KEY,
  aluno_id    INTEGER,
  plano_id    INTEGER,
  status      TEXT
);

```

3.4 Script de Inserção de Dados (População)

Insira os dados iniciais via **SQL Editor**:

```

INSERT INTO plano (nome)
VALUES ('Mensal'), ('Trimestral'), ('Anual');

```

3.5 Teste de Conexão

```

const supabase = require("../supabaseClient");

async function testarConexao() {
  const { data, error } = await supabase.from("pessoa").select("");
  if (error) console.error(error);
  else console.log(data);
}

testarConexao();

```

4. Execução do Backend

O backend consiste em um servidor **Node.js** responsável pelas APIs REST do sistema.

4.1 Inicialização

```
cd /home/alunos/a2539217/Documentos/GitHub/GymIO/backend  
npm start
```

Ou, caso não exista script configurado:

```
node server.js
```

O servidor será iniciado na porta definida no projeto (ex.: **3002**).

4.2 Rotas Disponíveis para Teste

As rotas podem ser testadas via navegador ou ferramentas como **Postman**:

- `http://localhost:3002/api/pessoas`
- `http://localhost:3002/api/pessoas?tipo=aluno`
- `http://localhost:3002/api/pessoas/planos`
- `http://localhost:3002/api/pessoas/total-alunos`

4.3 Modo de Desenvolvimento

```
npm run dev
```

Permite atualização automática durante alterações no código.

5. Execução do Frontend

O frontend é uma aplicação **React** construída com **Vite**.

5.1 Inicialização

```
cd /home/alunos/a2539217/Documentos/GitHub/GymIO/frontend  
npm run dev
```

A aplicação será disponibilizada em:

```
http://localhost:5173
```

5.2 Funcionalidades Disponíveis para Teste

- Cadastro de alunos
- Listagem de alunos
- Edição de dados

- Exclusão de registros
- Associação com planos

5.3 Build para Produção

```
npm run build
```

Os arquivos gerados podem ser hospedados em servidores como **Nginx** ou similares.